

Netflix – Data Exploration & Visualisation

Business Case Study

Objective: Analyze Netflix content to identify patterns that can help guide content production and business across countries.

Key business questions

- 1 . How is Netflix content split between Movies and TV Shows?
- 2 . How has content changed over the last 20 – 30 years?
- 3 . Has Netflix increased its focus on TV Shows in recent years?
- 4 . What content is available across countries?
- 5 . Which genres, ratings and content formats dominate?
- 6 . Which actors and directors appear most frequently?
- 7 . Which months have the highest number of TV Shows added?
- 8 . What actionable recommendations can be made for Netflix?

1 . Import libraries and load data

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

sns.set_theme(style="whitegrid")

df = pd.read_csv("netflix.csv")
print("Dataset shape:", df.shape)
df.head()
```

Dataset shape: (8807, 12)

Out[1]:	show_id	type	title	director	cast	country	date_added	release_year	rating
0	s 1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2020	PG-13
1	s 2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	September 24, 2021	2021	TV-MA
2	s 3	TV Show	Ganglands	Julien Leclercq	Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...	NaN	September 24, 2021	2021	TV-MA
3	s 4	TV Show	Jailbirds New Orleans	NaN	NaN	NaN	September 24, 2021	2021	TV-MA
4	s 5	TV Show	Kota Factory	NaN	Mayur More, Jitendra Kumar, Ranjan Raj, Alam K...	India	September 24, 2021	2021	TV-MA

2 . Basic data understanding

```
In [2]: print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
print("\nColumns:")
print(df.columns.tolist())
print("\nData types:")
print(df.dtypes)
print("\nInformation:")
df.info()
```

Rows: 8807
Columns: 12

Columns:
['show_id', 'type', 'title', 'director', 'cast', 'country', 'date_added',
'release_year', 'rating', 'duration', 'listed_in', 'description']

Data types:
show_id object
type object
title object
director object
cast object
country object
date_added object
release_year int64
rating object
duration object
listed_in object
description object
dtype: object

Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8807 entries, 0 to 8806
Data columns (total 12 columns):
Column Non-Null Count Dtype
--- -
0 show_id 8807 non-null object
1 type 8807 non-null object
2 title 8807 non-null object
3 director 6173 non-null object
4 cast 7982 non-null object
5 country 7976 non-null object
6 date_added 8797 non-null object
7 release_year 8807 non-null int64
8 rating 8803 non-null object
9 duration 8804 non-null object
10 listed_in 8807 non-null object
11 description 8807 non-null object
dtypes: int64(1), object(11)
memory usage: 825.8+ KB

3 . Missing value detection

```
In [3]: missing = pd.DataFrame({  
        "Missing Values": df.isnull().sum(),  
        "Missing %": (df.isnull().sum()/len(df)*100).round(2)  
    }).sort_values("Missing Values", ascending=False)  
missing
```

Out[3]:

	Missing Values	Missing %
director	2 6 3 4	2 9.9 1
country	8 3 1	9.4 4
cast	8 2 5	9.3 7
date_added	1 0	0.1 1
rating	4	0.0 5
duration	3	0.0 3
show_id	0	0.0 0
type	0	0.0 0
title	0	0.0 0
release_year	0	0.0 0
listed_in	0	0.0 0
description	0	0.0 0

Missing-value observation

`director` has the largest number of missing values, followed by `country` and `cast`. Because the descriptive attributes, deleting all missing rows would unnecessarily reduce the dataset. Missing categorical values will be represented as `Unknown` for analysis.

4 . Data preprocessing

```
In [4]: # Preserve raw data; create an analysis copy
netflix = df.copy()

# Fill missing categorical values
for col in ["director", "cast", "country", "rating", "duration"]:
    netflix[col] = netflix[col].fillna("Unknown")

# Date conversion
netflix["date_added"] = pd.to_datetime(netflix["date_added"], errors="coerce")
netflix["year_added"] = netflix["date_added"].dt.year
netflix["month_added"] = netflix["date_added"].dt.month

# Category conversion where appropriate
for col in ["type", "rating"]:
    netflix[col] = netflix[col].astype("category")

netflix.isnull().sum()
```

```
Out[4]: show_id      0
        type        0
        title       0
        director    0
        cast        0
        country     0
        date_added  98
        release_year 0
        rating      0
        duration    0
        listed_in   0
        description 0
        year_added  98
        month_added 98
        dtype: int64
```

5 . Statistical summary

```
In [5]: netflix.describe(include="all").T
```

Out[5]:

	count	unique	top	freq
show_id	8 8 0 7	8 8 0 7	s 1	1
type	8 8 0 7	2	Movie	6 1 3 1
title	8 8 0 7	8 8 0 7	Dick Johnson Is Dead	1
director	8 8 0 7	4 5 2 9	Unknown	2 6 3 4
cast	8 8 0 7	7 6 9 3	Unknown	8 2 5
country	8 8 0 7	7 4 9	United States	2 8 1 8
date_added	8 7 0 9	NaN	NaN	NaN
release_year	8 8 0 7.0	NaN	NaN	NaN
rating	8 8 0 7	1 8	TV-MA	3 2 0 7
duration	8 8 0 7	2 2 1	1 Season	1 7 9 3
listed_in	8 8 0 7	5 1 4	Dramas, International Movies	3 6 2
description	8 8 0 7	8 7 7 5	Paranormal activity at a lush, abandoned prope...	4
year_added	8 7 0 9.0	NaN	NaN	NaN
month_added	8 7 0 9.0	NaN	NaN	NaN

In [6]: `netflix["release_year"].describe()`

Out[6]:

```
count    8807.000000
mean     2014.180198
std       8.819312
min      1925.000000
25%      2013.000000
50%      2017.000000
75%      2019.000000
max       2021.000000
Name: release_year, dtype: float64
```

6 . Non-graphical analysis: unique values and value counts

In [7]:

```
print("Movie vs TV Show:\n", netflix["type"].value_counts())
print("\nPercentage:\n", (netflix["type"].value_counts(normalize=True)*10
print("\nUnique titles:", netflix["title"].nunique())
```

```
print("Unique release years:", netflix["release_year"].nunique())
print("\nTop ratings:\n", netflix["rating"].value_counts().head(10))
```

Movie vs TV Show:

```
type
Movie      6131
TV Show    2676
Name: count, dtype: int64
```

Percentage:

```
type
Movie      69.62
TV Show    30.38
Name: proportion, dtype: float64
```

Unique titles: 8807

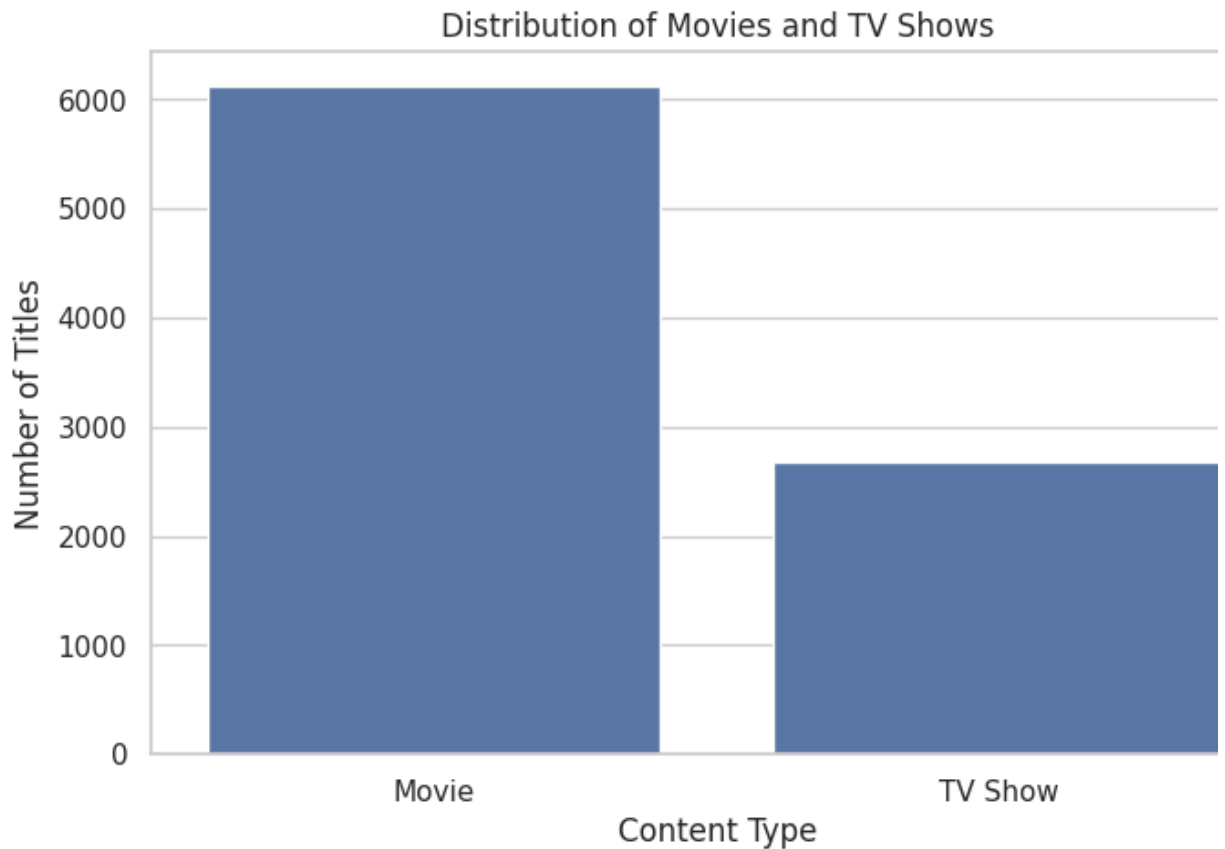
Unique release years: 74

Top ratings:

```
rating
TV-MA    3207
TV-14    2160
TV-PG     863
R         799
PG-13     490
TV-Y7     334
TV-Y      307
PG        287
TV-G      220
NR         80
Name: count, dtype: int64
```

7 . Univariate analysis – content type

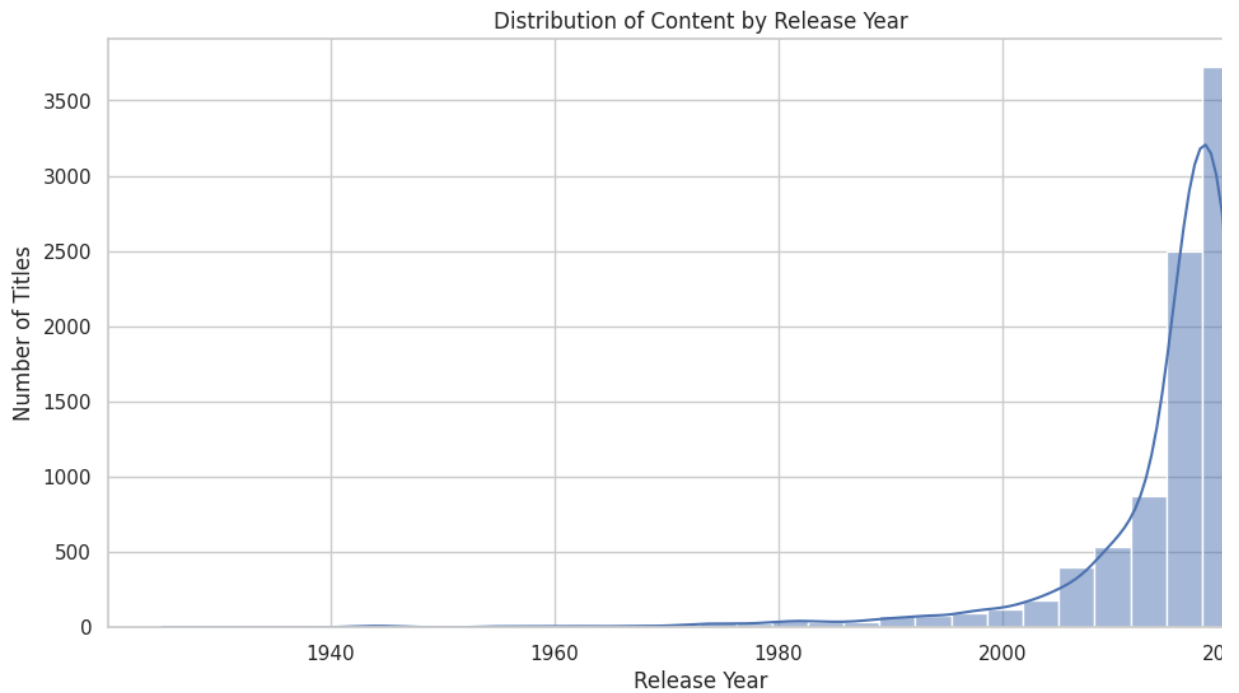
```
In [8]: plt.figure(figsize=(8,5))
sns.countplot(data=netflix, x="type", color="C0")
plt.title("Distribution of Movies and TV Shows")
plt.xlabel("Content Type")
plt.ylabel("Number of Titles")
plt.show()
```



Observation: Movies are the majority of the catalog, accounting for about **69.6%** of titles, while TV account for about **30.4%**.

8 . Release-year distribution

```
In [9]: plt.figure(figsize=(12,6))
sns.histplot(data=netflix, x="release_year", bins=30, kde=True, color="C0
plt.title("Distribution of Content by Release Year")
plt.xlabel("Release Year")
plt.ylabel("Number of Titles")
plt.show()
```

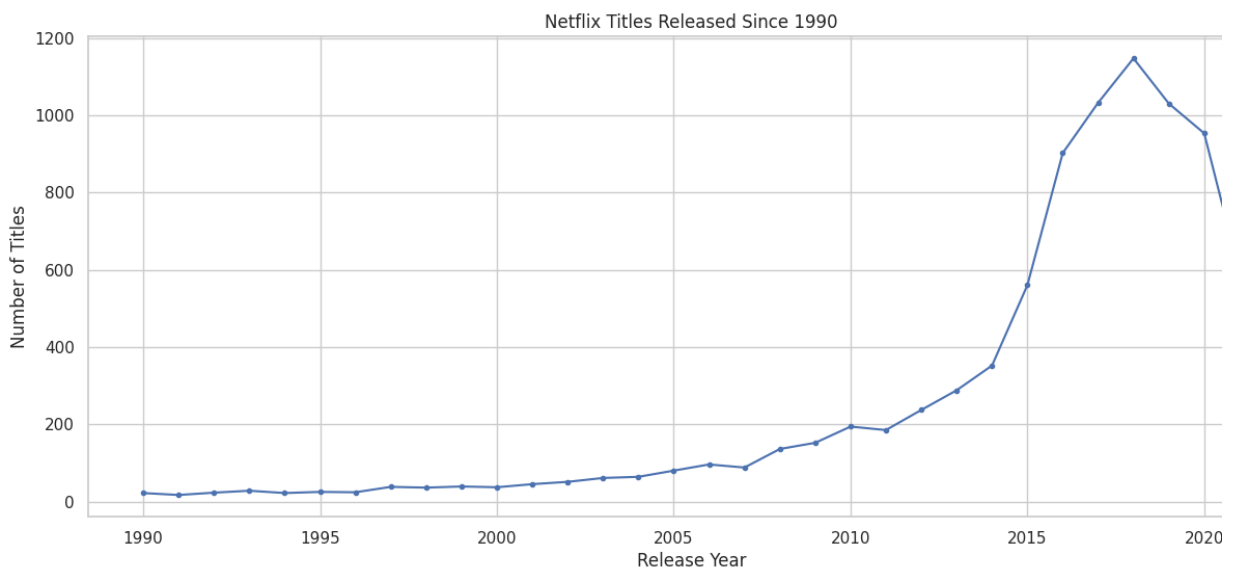


Observation: The distribution is strongly concentrated toward recent releases, especially from the mid- 2010s onward. The median release year is 2017.

9. Content released per year – last 30 years

```
In [10]: year_counts = netflix["release_year"].value_counts().sort_index()
recent_years = year_counts[year_counts.index >= 1990]

plt.figure(figsize=(15,6))
plt.plot(recent_years.index, recent_years.values, marker="o", markersize=10)
plt.title("Netflix Titles Released Since 1990")
plt.xlabel("Release Year")
plt.ylabel("Number of Titles")
plt.show()
```

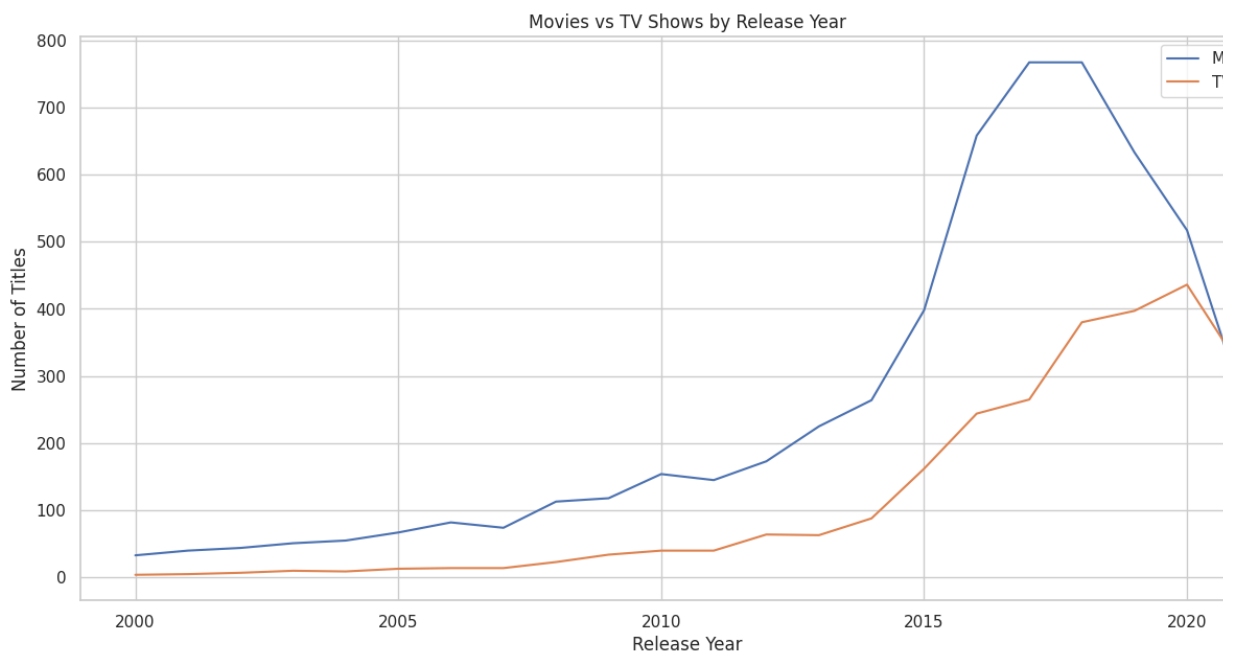


1 0 . Movies vs TV Shows by release year

```
In [11]: type_year = pd.crosstab(netflix["release_year"], netflix["type"])
type_year = type_year[type_year.index >= 2000]

plt.figure(figsize=(15,7))
for c in ["Movie", "TV Show"]:
    plt.plot(type_year.index, type_year[c], label=c)
plt.title("Movies vs TV Shows by Release Year")
plt.xlabel("Release Year")
plt.ylabel("Number of Titles")
plt.legend()
plt.show()

print(type_year.loc[2017:2021])
```



type	Movie	TV Show
release_year		
2017	767	265
2018	767	380
2019	633	397
2020	517	436
2021	277	315

Observation: Movies remain larger overall, but TV Shows have grown faster in the recent period. In the d. Shows exceed Movies for release year 2 0 2 1 (3 1 5 vs 2 7 7), suggesting a strong emphasis on episodic content.

1 1 . Has Netflix focused more on TV Shows in recent year

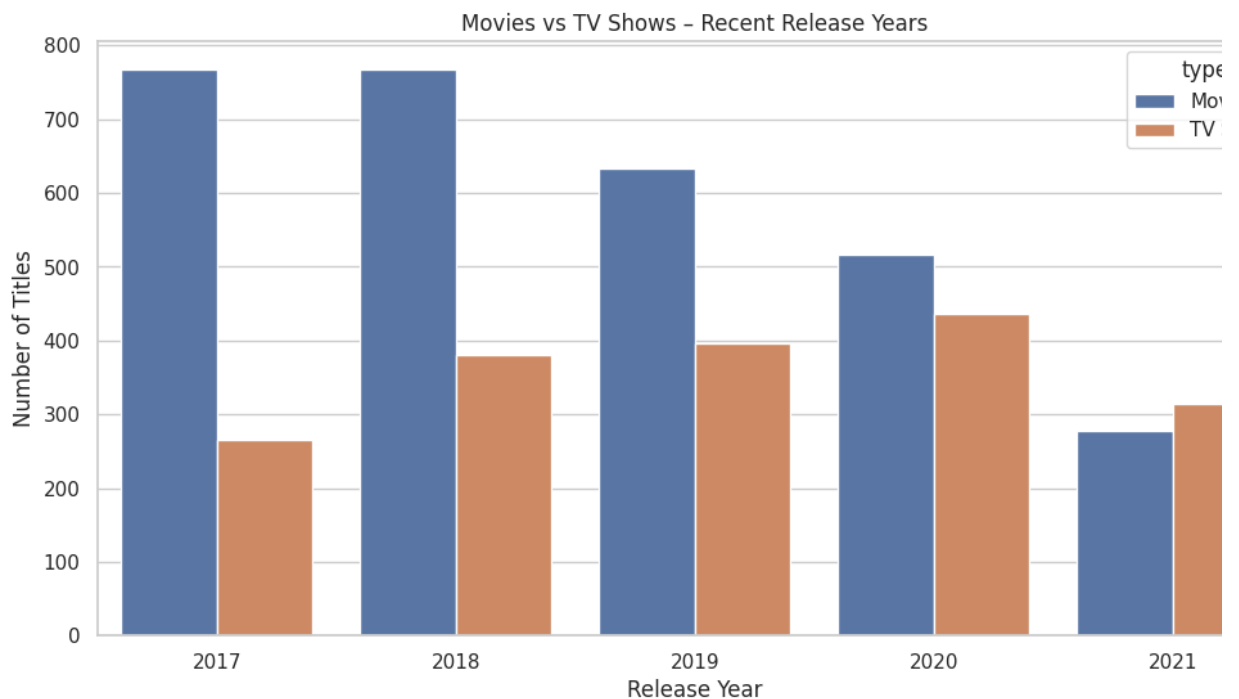
```
In [12]: recent = netflix[netflix["release_year"] >= 2017]
print(recent["type"].value_counts())
print("\nPercentages:\n", (recent["type"].value_counts(normalize=True)*10

plt.figure(figsize=(12,6))
```

```
sns.countplot(data=recent, x="release_year", hue="type", palette="deep")
plt.title("Movies vs TV Shows – Recent Release Years")
plt.xlabel("Release Year")
plt.ylabel("Number of Titles")
plt.show()
```

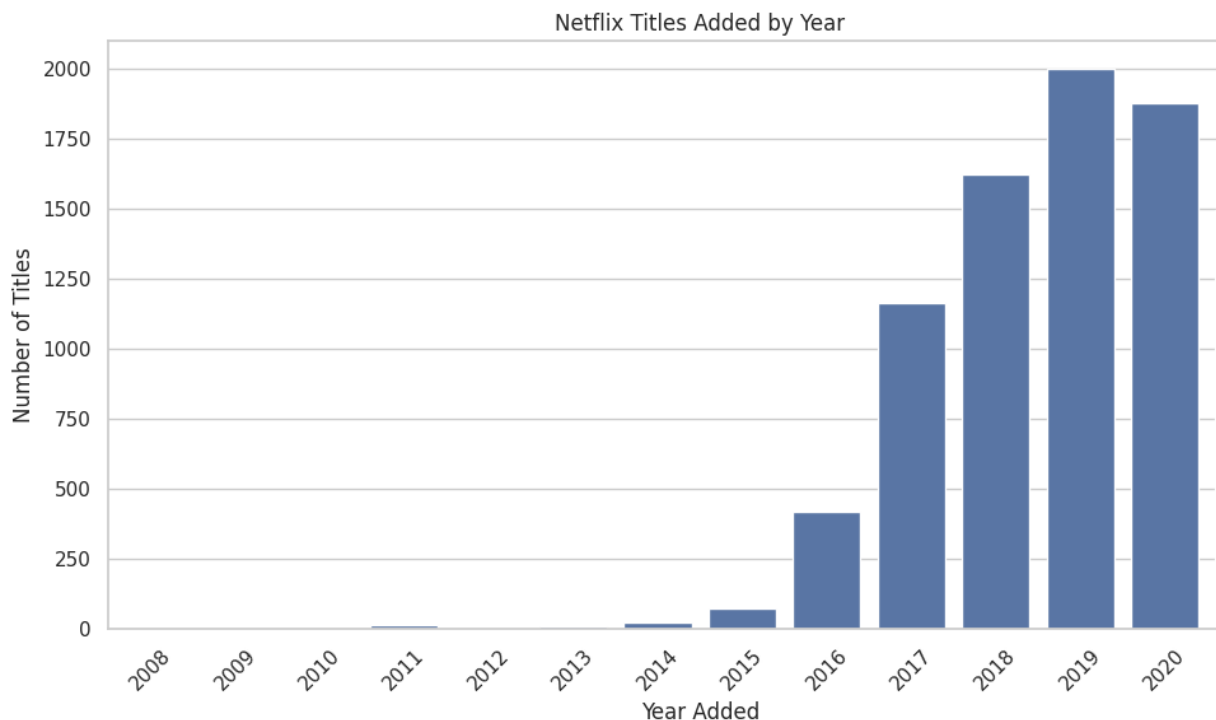
```
type
Movie      2961
TV Show    1793
Name: count, dtype: int64
```

```
Percentages:
type
Movie      62.28
TV Show    37.72
Name: proportion, dtype: float64
```



1 2 . Date-added analysis

```
In [13]: added_year_counts = netflix["year_added"].value_counts().sort_index()
plt.figure(figsize=(12,6))
sns.barplot(x=added_year_counts.index.astype(int), y=added_year_counts.values)
plt.title("Netflix Titles Added by Year")
plt.xlabel("Year Added")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.show()
print(added_year_counts)
```



```

year_added
2008.0      2
2009.0      2
2010.0      1
2011.0     13
2012.0      3
2013.0     10
2014.0     23
2015.0     73
2016.0    418
2017.0   1164
2018.0   1625
2019.0   1999
2020.0   1878
2021.0   1498
Name: count, dtype: int64

```

1 3 . Best month to launch/add a TV Show

```

In [14]: month_names = {1:"January",2:"February",3:"March",4:"April",5:"May",6:"Ju

tv = netflix[netflix["type"] == "TV Show"].copy()
tv_month = tv["month_added"].value_counts().sort_index()
tv_month_named = tv_month.rename(index=month_names)
print(tv_month_named)

plt.figure(figsize=(12,6))
sns.barplot(x=tv_month_named.index, y=tv_month_named.values, color="C0")
plt.title("TV Shows Added by Month")
plt.xlabel("Month")
plt.ylabel("Number of TV Shows Added")
plt.xticks(rotation=45)
plt.show()

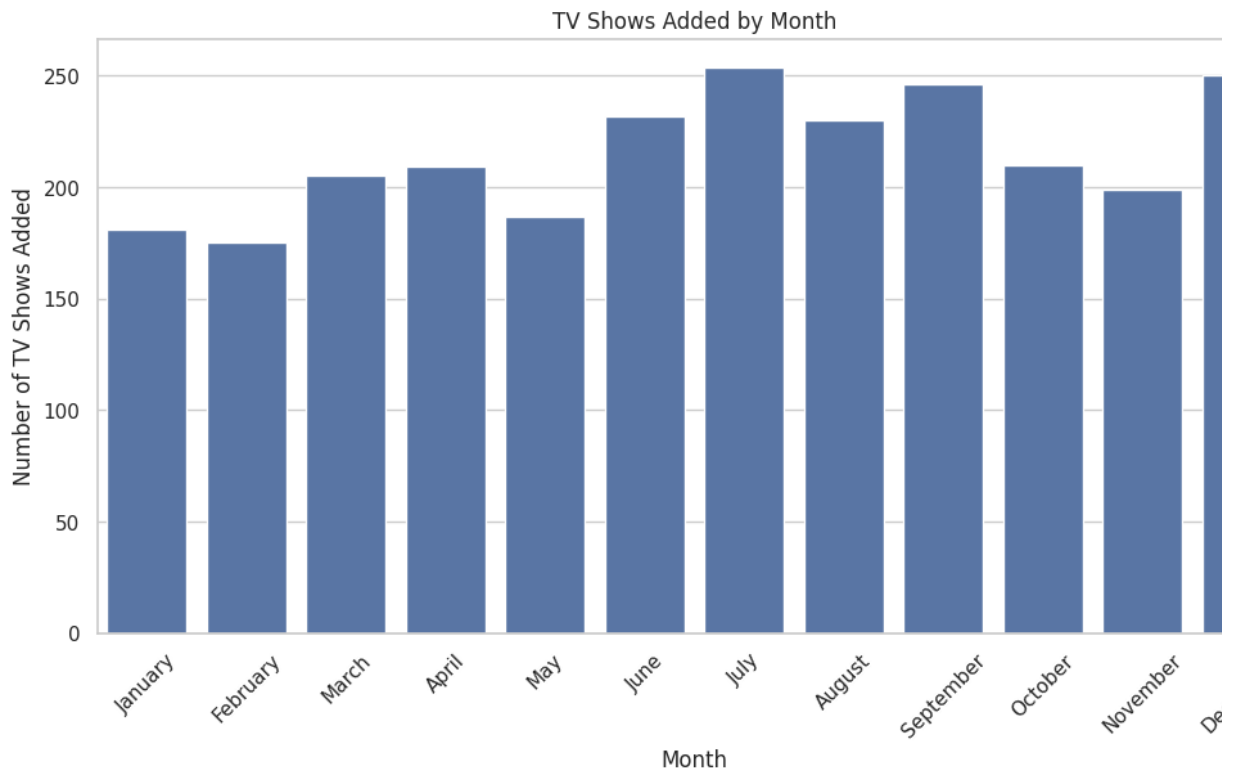
print("Highest month:", tv_month_named.idxmax(), "with", tv_month_named.m

```

```

month_added
January      181
February     175
March        205
April        209
May          187
June         232
July         254
August       230
September   246
October     210
November    199
December    250
Name: count, dtype: int64

```



Highest month: July with 254 TV Shows

Business caveat: July has the highest number of TV Shows added (2 5 4). This indicates the most common historical addition period in this dataset, but the dataset does not contain viewership, engagement or subscription so it cannot prove that July produces the best business performance.

1 4 . Country analysis – unnesting

```

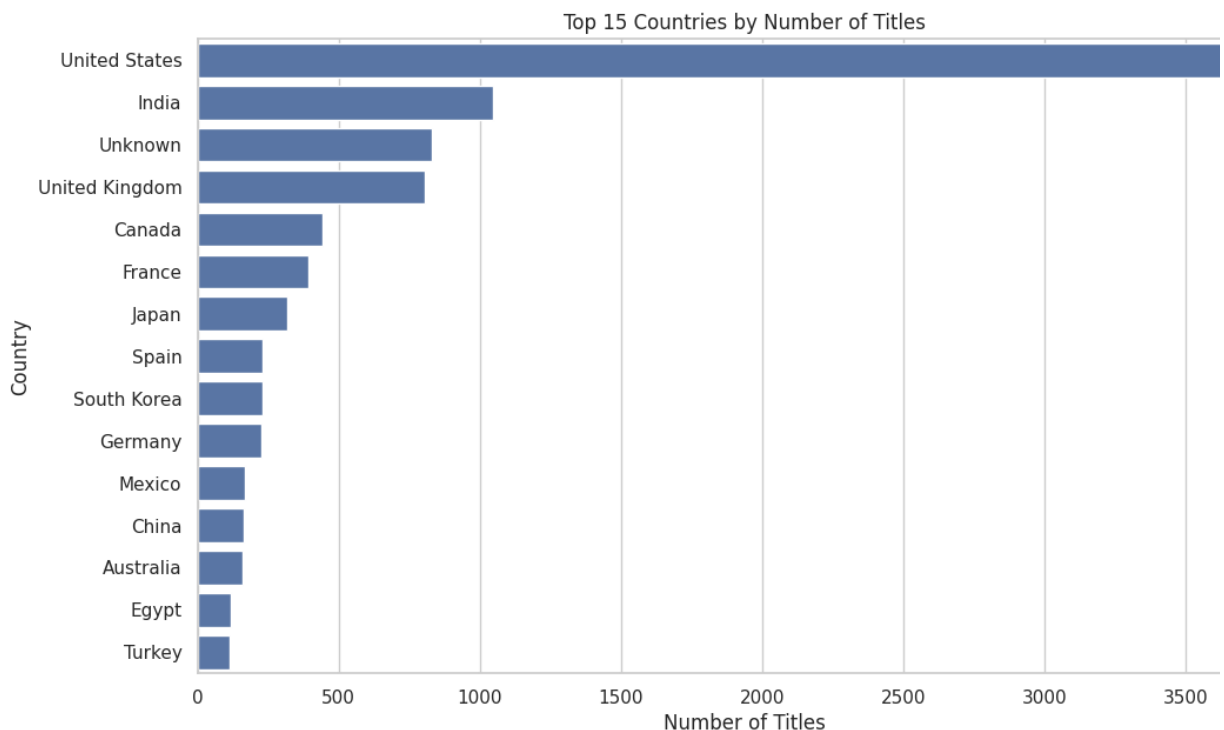
In [15]: country_df = netflix.copy()
country_df["country"] = country_df["country"].str.split(", ")
country_df = country_df.explode("country")
country_counts = country_df["country"].value_counts()
print(country_counts.head(15))

plt.figure(figsize=(12,7))
top15 = country_counts.head(15)
sns.barplot(x=top15.values, y=top15.index, color="C0")
plt.title("Top 15 Countries by Number of Titles")
plt.xlabel("Number of Titles")

```

```
plt.ylabel("Country")
plt.show()
```

```
country
United States    3689
India            1046
Unknown          831
United Kingdom   804
Canada           445
France           393
Japan            318
Spain            232
South Korea      231
Germany          226
Mexico           169
China            162
Australia        160
Egypt            117
Turkey           113
Name: count, dtype: int64
```

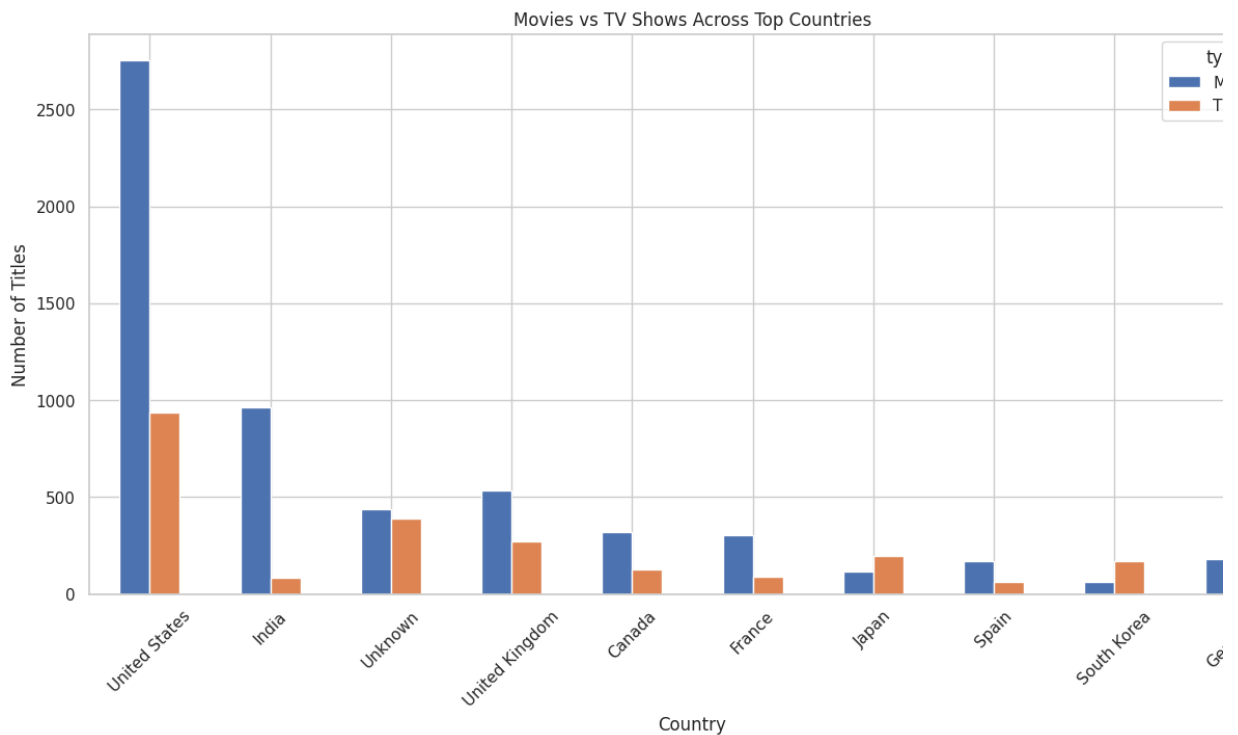


1 5 . Country × content type

```
In [16]: country_type = pd.crosstab(country_df["country"], country_df["type"])
country_type["Total"] = country_type.sum(axis=1)
country_type = country_type.sort_values("Total", ascending=False)
print(country_type.head(15))

top = country_type.head(10).drop(columns="Total")
top.plot(kind="bar", figsize=(15,7))
plt.title("Movies vs TV Shows Across Top Countries")
plt.xlabel("Country")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.show()
```

type	Movie	TV Show	Total
country			
United States	2751	938	3689
India	962	84	1046
Unknown	440	391	831
United Kingdom	532	272	804
Canada	319	126	445
France	303	90	393
Japan	119	199	318
Spain	171	61	232
South Korea	61	170	231
Germany	182	44	226
Mexico	111	58	169
China	114	48	162
Australia	94	66	160
Egypt	102	15	117
Turkey	83	30	113



1 6 . Genre analysis – unnesting

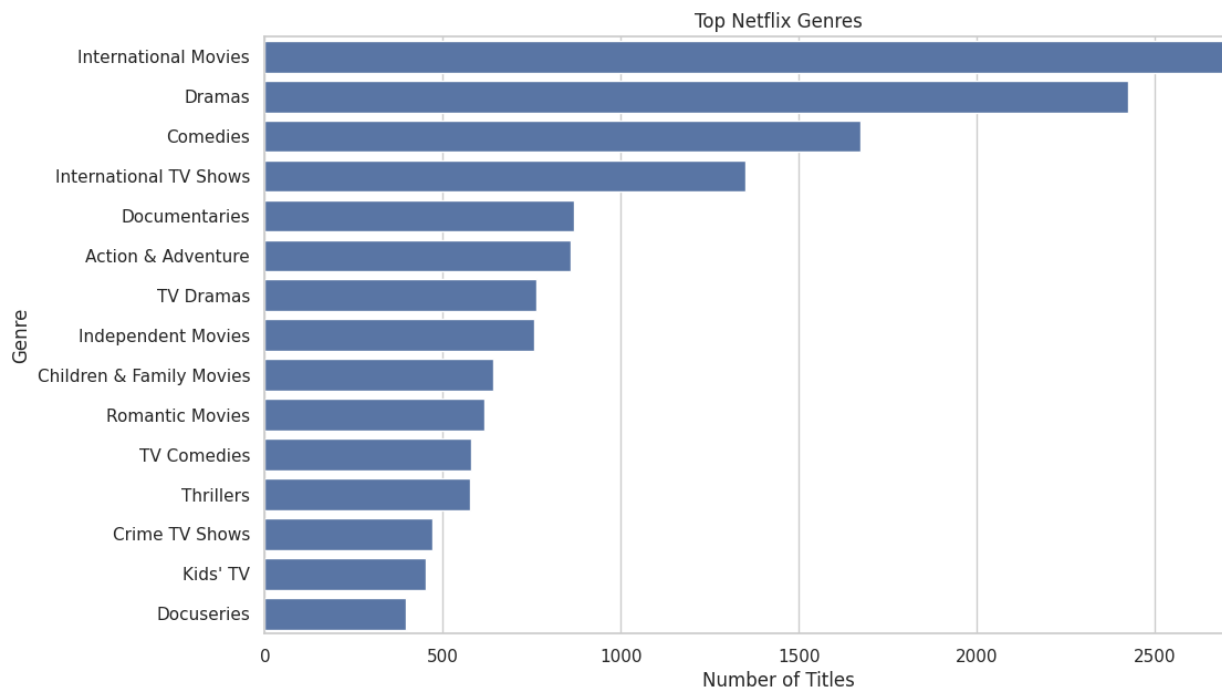
```
In [17]: genre_df = netflix.copy()
genre_df["listed_in"] = genre_df["listed_in"].str.split(", ")
genre_df = genre_df.explode("listed_in")
genre_counts = genre_df["listed_in"].value_counts()
print(genre_counts.head(15))

top_genres = genre_counts.head(15)
plt.figure(figsize=(12,7))
sns.barplot(x=top_genres.values, y=top_genres.index, color="C0")
plt.title("Top Netflix Genres")
plt.xlabel("Number of Titles")
plt.ylabel("Genre")
plt.show()
```

```

listed_in
International Movies      2752
Dramas                   2427
Comedies                 1674
International TV Shows   1351
Documentaries            869
Action & Adventure       859
TV Dramas               763
Independent Movies       756
Children & Family Movies 641
Romantic Movies          616
TV Comedies              581
Thrillers                577
Crime TV Shows           470
Kids' TV                 451
Docuseries               395
Name: count, dtype: int64

```

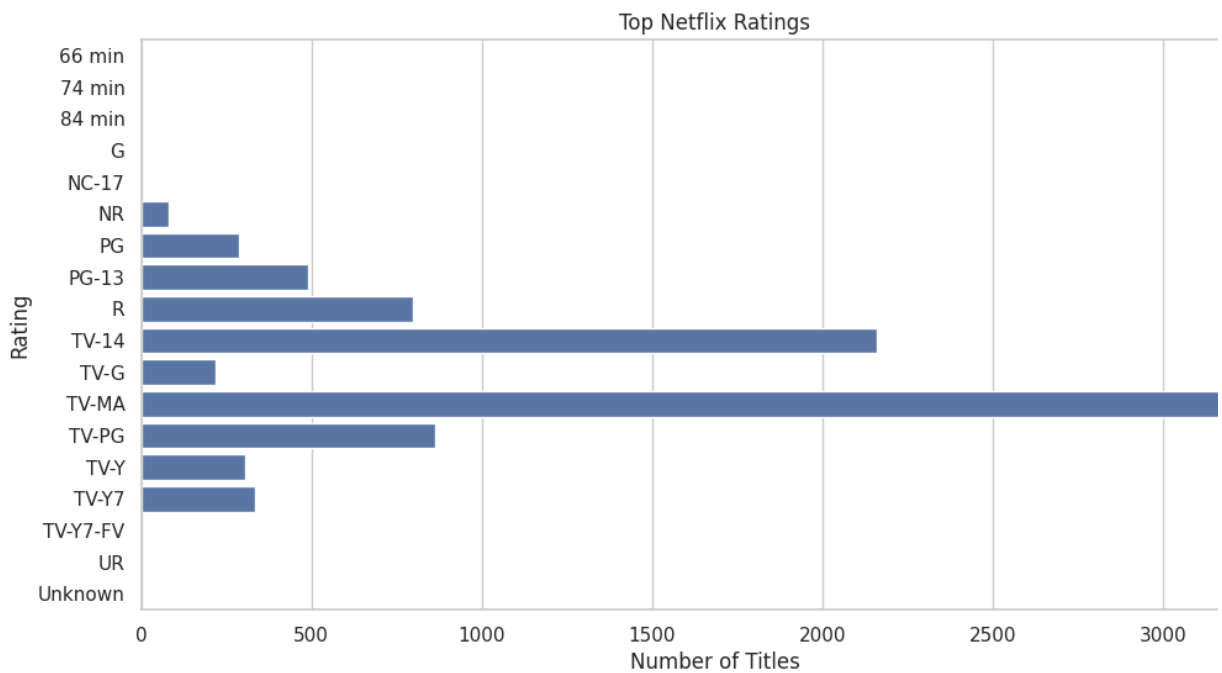


1 7 . Ratings

```

In [18]: rating_counts = netflix["rating"].value_counts()
plt.figure(figsize=(12,6))
r10 = rating_counts.head(10)
sns.barplot(x=r10.values, y=r10.index, color="C0")
plt.title("Top Netflix Ratings")
plt.xlabel("Number of Titles")
plt.ylabel("Rating")
plt.show()
print(rating_counts.head(10))

```



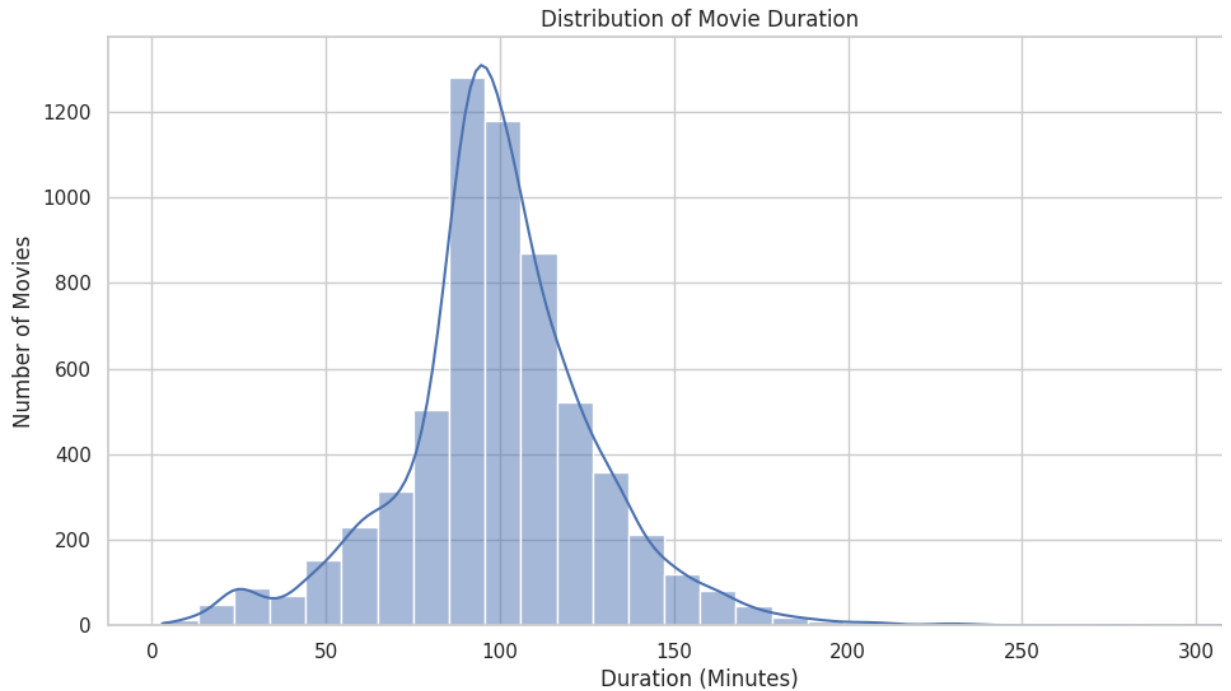
```
rating
TV-MA    3207
TV-14    2160
TV-PG     863
R         799
PG-13     490
TV-Y7     334
TV-Y      307
PG         287
TV-G      220
NR         80
Name: count, dtype: int64
```

1 8 . Movie duration analysis

```
In [19]: movies = netflix[netflix["type"] == "Movie"].copy()
movies["duration_minutes"] = pd.to_numeric(movies["duration"].str.extract(
print(movies["duration_minutes"].describe())

plt.figure(figsize=(12,6))
sns.histplot(movies["duration_minutes"].dropna(), bins=30, kde=True, color=
plt.title("Distribution of Movie Duration")
plt.xlabel("Duration (Minutes)")
plt.ylabel("Number of Movies")
plt.show()
```

```
count    6128.000000
mean      99.577187
std       28.290593
min        3.000000
25%       87.000000
50%       98.000000
75%      114.000000
max      312.000000
Name: duration_minutes, dtype: float64
```

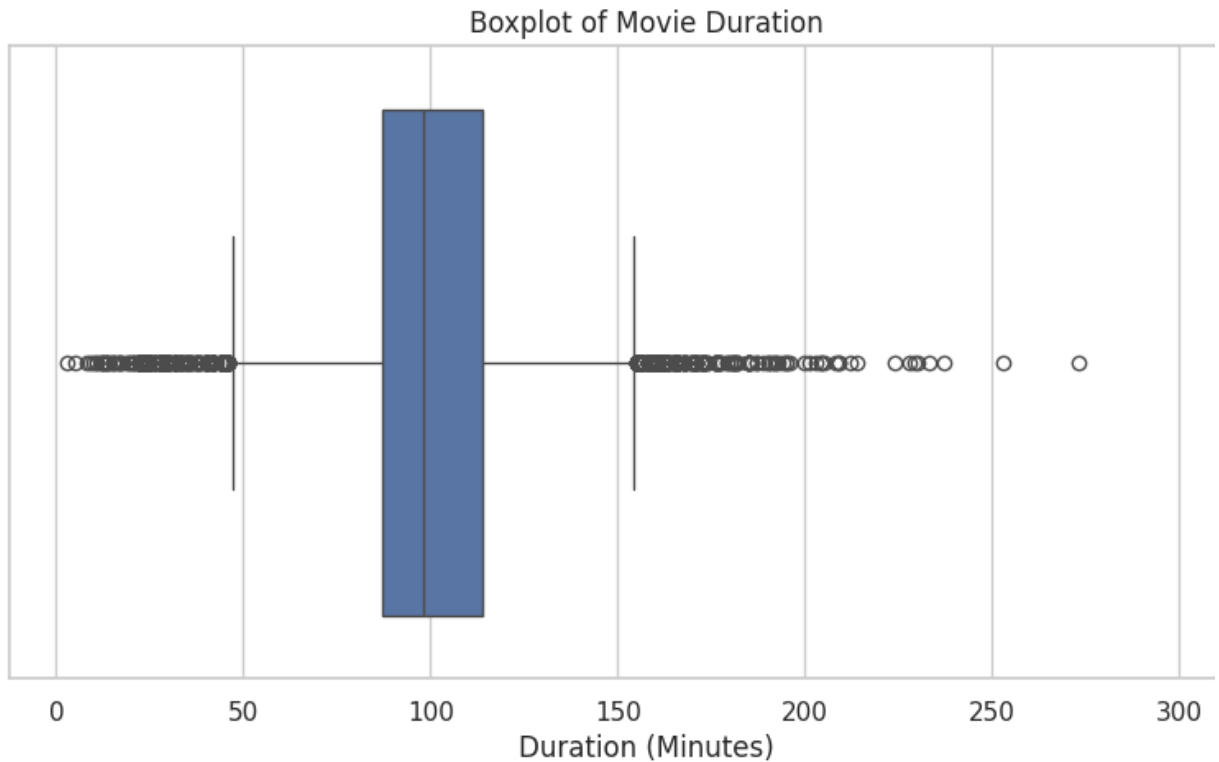


Observation: Movie duration has a median of 98 minutes; the middle 50% lies between 81 and 114 minutes. Most movies therefore fall into a moderate-length range.

19. Boxplot – movie duration

```
In [20]: plt.figure(figsize=(10,5))
sns.boxplot(x=movies["duration_minutes"].dropna(), color="C0")
plt.title("Boxplot of Movie Duration")
plt.xlabel("Duration (Minutes)")
plt.show()

q1 = movies["duration_minutes"].quantile(0.25)
q3 = movies["duration_minutes"].quantile(0.75)
iqr = q3 - q1
lower = q1 - 1.5*iqr
upper = q3 + 1.5*iqr
outliers = movies[(movies["duration_minutes"] < lower) | (movies["duration_minutes"] > upper)]
print(f"IQR: {iqr:.1f} minutes")
print(f"Lower bound: {lower:.1f}, Upper bound: {upper:.1f}")
print("Potential movie-duration outliers:", len(outliers))
```



IQR: 27.0 minutes
 Lower bound: 46.5, Upper bound: 154.5
 Potential movie-duration outliers: 450

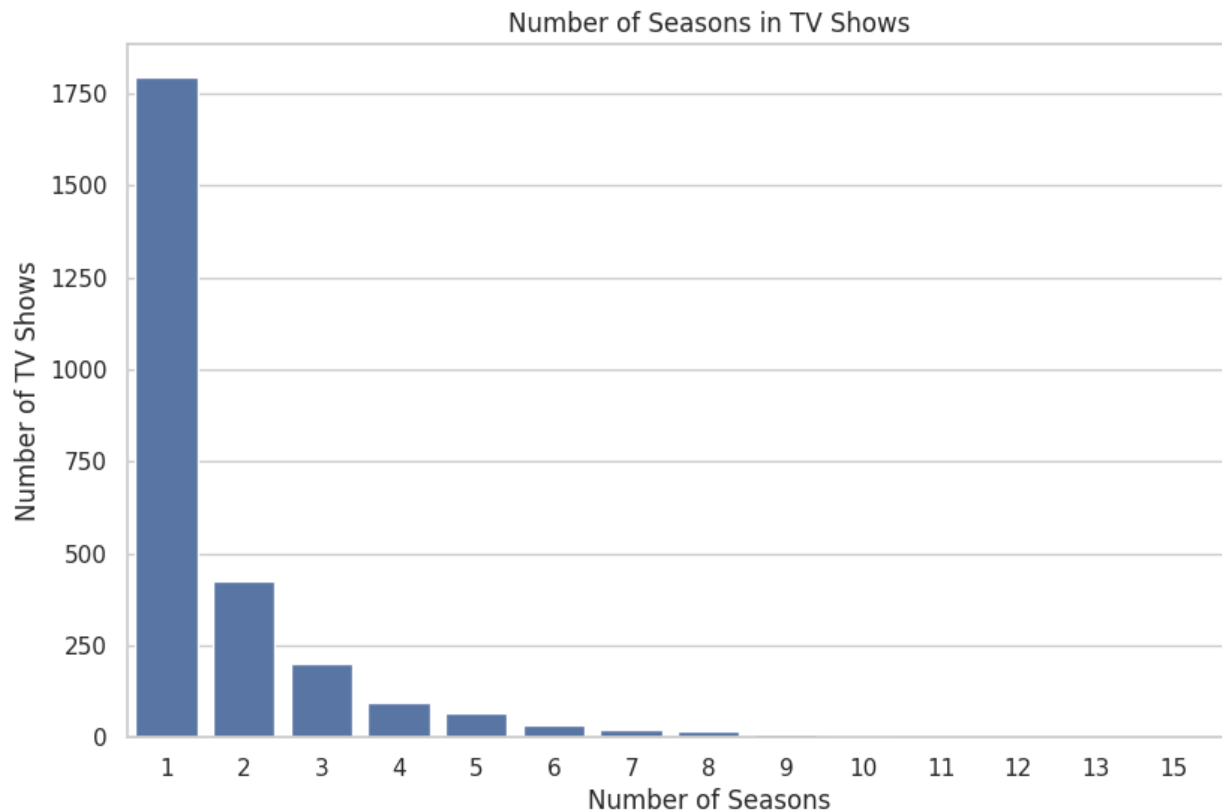
2 0 . TV Show seasons analysis

```
In [21]: tv = netflix[netflix["type"] == "TV Show"].copy()
tv["seasons"] = pd.to_numeric(tv["duration"].str.extract(r"(\d+)")[0], errors="coerce")
print(tv["seasons"].describe())

plt.figure(figsize=(10,6))
sns.countplot(data=tv, x="seasons", color="C0")
plt.title("Number of Seasons in TV Shows")
plt.xlabel("Number of Seasons")
plt.ylabel("Number of TV Shows")
plt.show()
```

count	2676.000000
mean	1.764948
std	1.582752
min	1.000000
25%	1.000000
50%	1.000000
75%	2.000000
max	17.000000

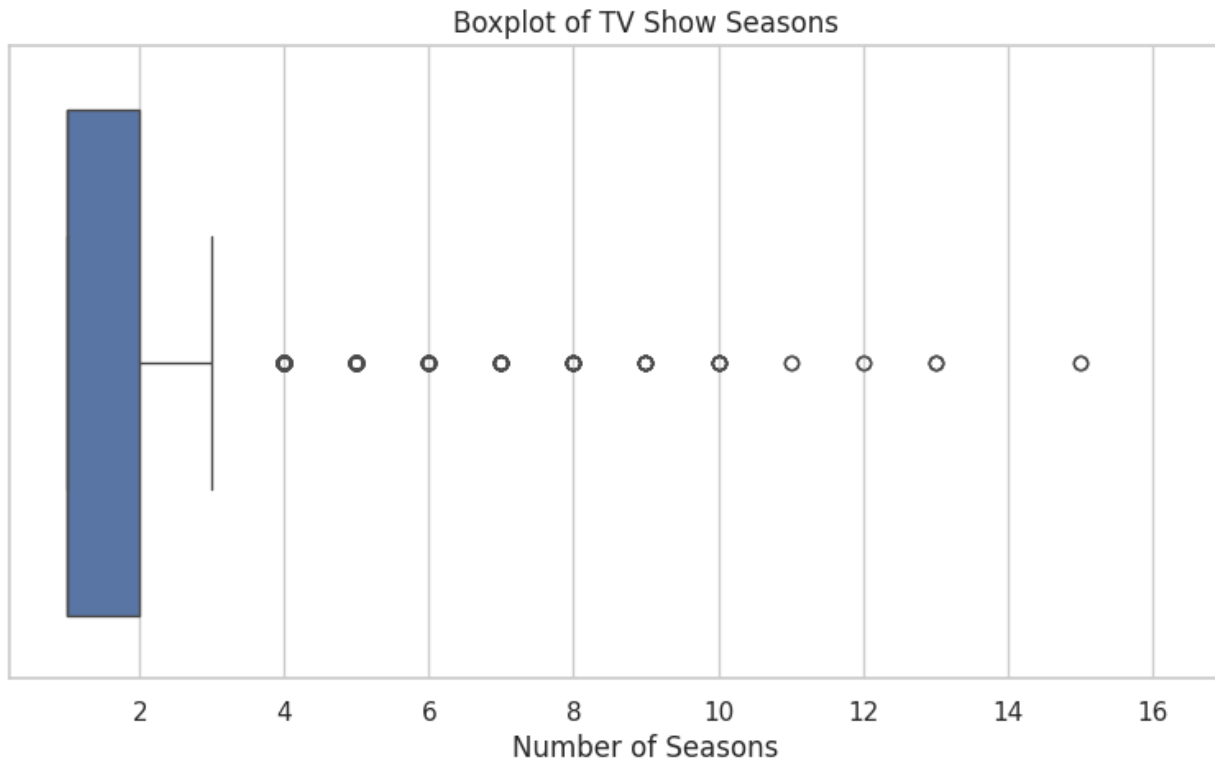
Name: seasons, dtype: float64



2 1 . Boxplot – TV Show seasons

```
In [22]: plt.figure(figsize=(10,5))
sns.boxplot(x=tv["seasons"].dropna(), color="C0")
plt.title("Boxplot of TV Show Seasons")
plt.xlabel("Number of Seasons")
plt.show()

q1 = tv["seasons"].quantile(0.25)
q3 = tv["seasons"].quantile(0.75)
iqr = q3-q1
upper = q3+1.5*iqr
print(f"Q1={q1}, Median={tv['seasons'].median()}, Q3={q3}, IQR={iqr}")
print("Potential high-season outliers:", (tv["seasons"]>upper).sum())
```



Q1=1.0, Median=1.0, Q3=2.0, IQR=1.0
 Potential high-season outliers: 259

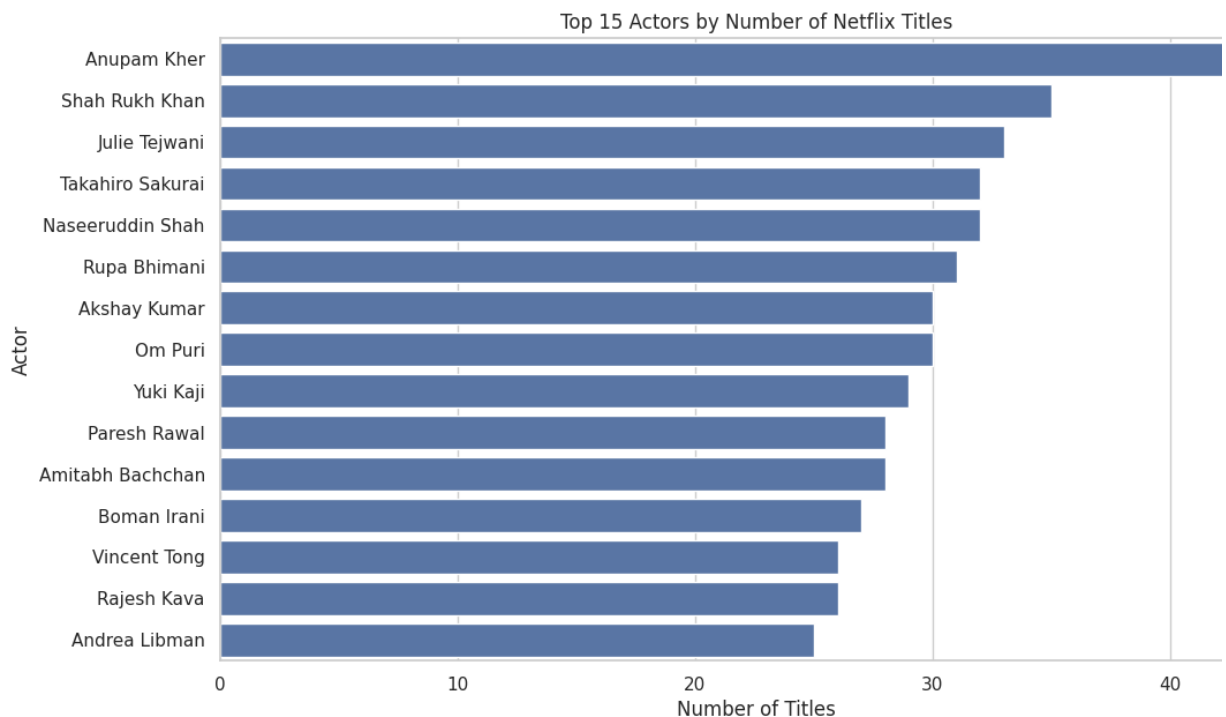
2.2 Actors – unnesting and frequency

```
In [23]: actor_df = netflix.copy()
actor_df["cast"] = actor_df["cast"].str.split(", ")
actor_df = actor_df.explode("cast")
actor_counts = actor_df[actor_df["cast"] != "Unknown"]["cast"].value_counts()
print(actor_counts.head(15))

plt.figure(figsize=(12,7))
a15 = actor_counts.head(15)
sns.barplot(x=a15.values, y=a15.index, color="C0")
plt.title("Top 15 Actors by Number of Netflix Titles")
plt.xlabel("Number of Titles")
plt.ylabel("Actor")
plt.show()
```

cast	
Anupam Kher	43
Shah Rukh Khan	35
Julie Tejwani	33
Takahiro Sakurai	32
Naseeruddin Shah	32
Rupa Bhimani	31
Akshay Kumar	30
Om Puri	30
Yuki Kaji	29
Paresh Rawal	28
Amitabh Bachchan	28
Boman Irani	27
Vincent Tong	26
Rajesh Kava	26
Andrea Libman	25

Name: count, dtype: int64



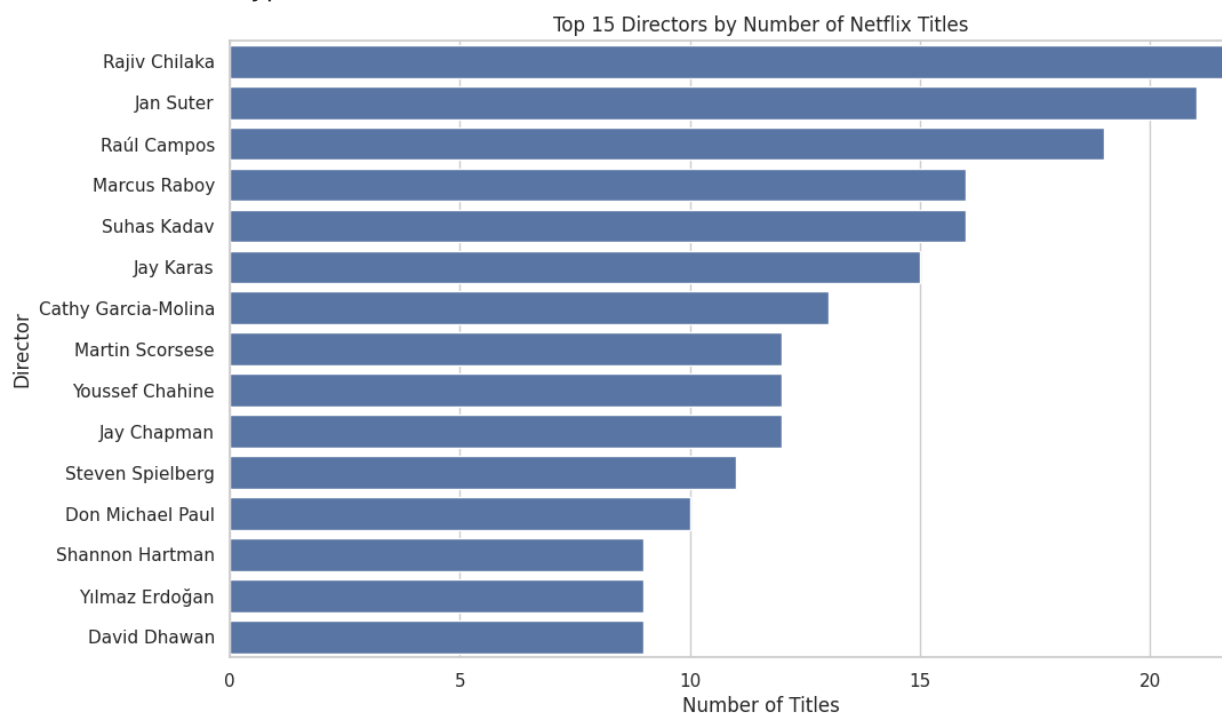
2 3 . Directors – unnesting and frequency

```
In [24]: director_df = netflix.copy()
director_df["director"] = director_df["director"].str.split(", ")
director_df = director_df.explode("director")
director_counts = director_df[director_df["director"] != "Unknown"]["director"].value_counts()
print(director_counts.head(15))

plt.figure(figsize=(12,7))
d15 = director_counts.head(15)
sns.barplot(x=d15.values, y=d15.index, color="C0")
plt.title("Top 15 Directors by Number of Netflix Titles")
plt.xlabel("Number of Titles")
plt.ylabel("Director")
plt.show()
```

director	
Rajiv Chilaka	22
Jan Suter	21
Raúl Campos	19
Marcus Raboy	16
Suhas Kadav	16
Jay Karas	15
Cathy Garcia-Molina	13
Martin Scorsese	12
Youssef Chahine	12
Jay Chapman	12
Steven Spielberg	11
Don Michael Paul	10
Shannon Hartman	9
Yılmaz Erdoğan	9
David Dhawan	9

Name: count, dtype: int64

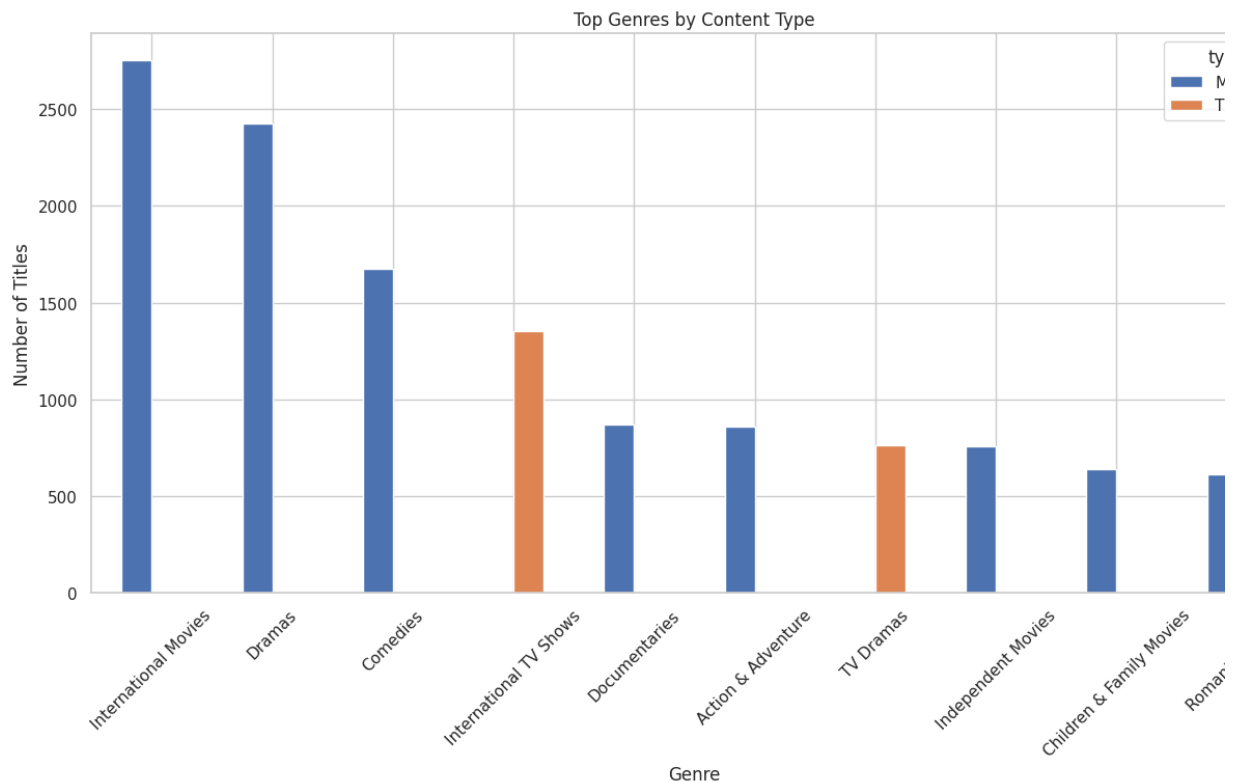


2 4 . Genre × content type

```
In [25]: genre_type = pd.crosstab(genre_df["listed_in"], genre_df["type"])
genre_type["Total"] = genre_type.sum(axis=1)
genre_type = genre_type.sort_values("Total", ascending=False)
print(genre_type.head(15))

gt = genre_type.head(10).drop(columns="Total")
gt.plot(kind="bar", figsize=(15,7))
plt.title("Top Genres by Content Type")
plt.xlabel("Genre")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.show()
```

type	Movie	TV Show	Total
listed_in			
International Movies	2752	0	2752
Dramas	2427	0	2427
Comedies	1674	0	1674
International TV Shows	0	1351	1351
Documentaries	869	0	869
Action & Adventure	859	0	859
TV Dramas	0	763	763
Independent Movies	756	0	756
Children & Family Movies	641	0	641
Romantic Movies	616	0	616
TV Comedies	0	581	581
Thrillers	577	0	577
Crime TV Shows	0	470	470
Kids' TV	0	451	451
Docuseries	0	395	395

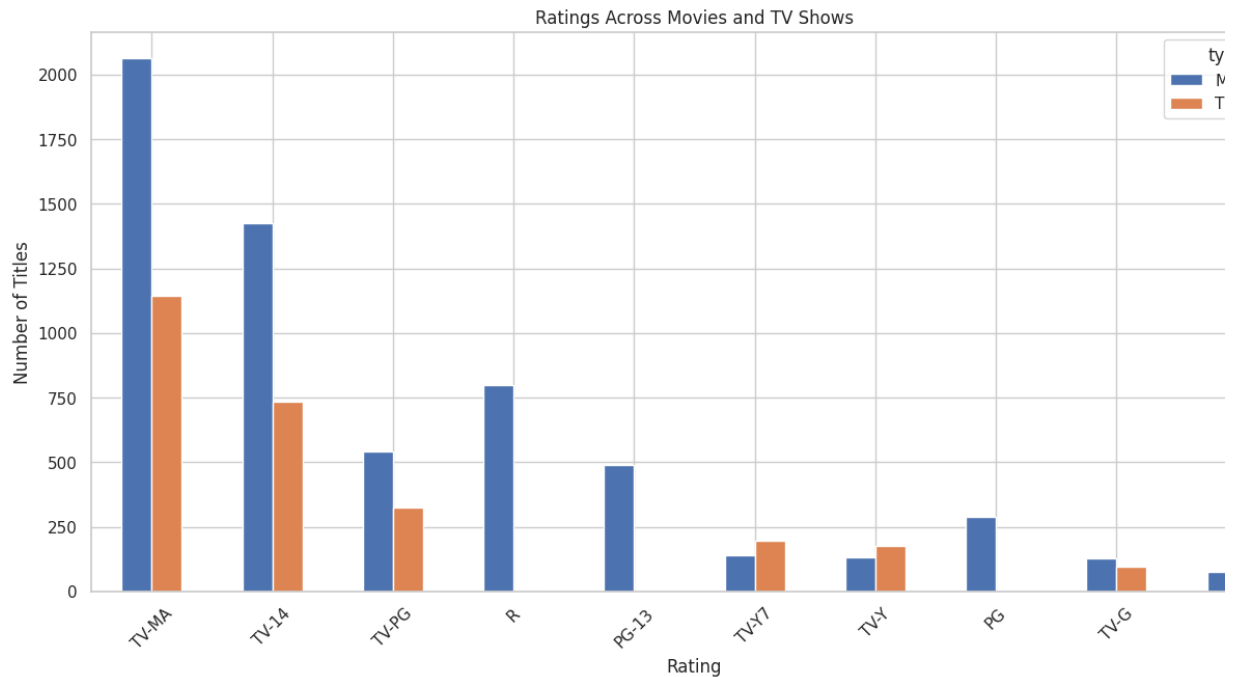


2.5. Rating × content type

```
In [26]: rating_type = pd.crosstab(netflix["rating"], netflix["type"])
print(rating_type.sort_values("Movie", ascending=False).head(10))

rt = rating_type.loc[rating_type.sum(axis=1).sort_values(ascending=False)]
rt.plot(kind="bar", figsize=(15,7))
plt.title("Ratings Across Movies and TV Shows")
plt.xlabel("Rating")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.show()
```

type	Movie	TV Show
rating		
TV-MA	2062	1145
TV-14	1427	733
R	797	2
TV-PG	540	323
PG-13	490	0
PG	287	0
TV-Y7	139	195
TV-Y	131	176
TV-G	126	94
NR	75	5



2 6 . Additional outlier and data-quality checks

```
In [27]: print("Duplicate rows:", netflix.duplicated().sum())
print("Duplicate show IDs:", netflix["show_id"].duplicated().sum())
print("Release-year range:", netflix["release_year"].min(), "to", netflix["release_year"].max())
print("Movie duration min/max:", movies["duration_minutes"].min(), movies["duration_minutes"].max())
print("TV seasons min/max:", tv["seasons"].min(), tv["seasons"].max())
```

```
Duplicate rows: 0
Duplicate show IDs: 0
Release-year range: 1925 to 2021
Movie duration min/max: 3.0 312.0
TV seasons min/max: 1 17
```

2 7 . Insights based on the analysis

Range and distribution

- Release years range from 1 9 2 5 to 2 0 2 1 , with a median of 2 0 1 7 .
- Movie duration ranges from 3 to 3 1 2 minutes, with a median of 9 8 minutes.
- TV Shows range from 1 to 1 7 seasons, with a median of 1 season.

- Movies make up about **69.6%** of all titles; TV Shows make up about **30.4%**.

Missing values

- Director has the most missing observations (**2,634**), followed by country (**831**) and cast (**825**).
- Missing descriptive fields should not be treated as evidence that the corresponding content does not exist; they simply indicate unavailable metadata.

Content mix

- Movies dominate the catalog overall.
- The gap between Movies and TV Shows narrows considerably in recent release years.
- TV Shows exceed Movies for release year **2021** in the available dataset (**315** vs **2**).

Countries

- The United States (**3,689**) is the largest source of content.
- India (**1,046**) is second, showing strong international production potential.
- The United Kingdom, Canada, France, Japan, Spain and South Korea are also major contributors.

Genres and ratings

- International Movies, Dramas, Comedies and International TV Shows are among the largest genre categories.
- TV-MA and TV-**14** are the two most common ratings, indicating a strong mature/teen audience.

Launch timing

- July is the month with the highest number of TV Shows added (**254**), followed by December (**246**) and September (**246**).
- This is a catalog-addition pattern, not a proof of audience or revenue performance.

Actors and directors

- Anupam Kher is the most frequently appearing actor in the full cast-expanded dataset (**43** titles).
- Rajiv Chilaka is the most frequently appearing director (**22** titles).
- These frequencies describe catalog presence; they should not be interpreted as measures of quality or profitability without viewer-performance data.

2 8 . Business Insights

- 1 . Netflix still has a movie-heavy catalog, but the recent release mix shows a meaningful shift toward TV
- 2 . Recent content is strategically important, with the catalog heavily concentrated in releases from the mid- 2 0 1 0 s onward.
- 3 . International production is essential to Netflix's growth, with strong contributions from India, the UK, France, Japan, Spain and South Korea.
- 4 . Local-language and international content are major parts of the catalog, not a small niche.
- 5 . Drama and comedy have broad representation, making them useful categories for both global and local storytelling.
- 6 . Most TV Shows are short-running, with a median of one season. Netflix has significant experience with limited-series formats.
- 7 . Most Movies are around 9 0 – 1 2 0 minutes, providing a practical benchmark for standard length productions.
- 8 . The dataset does not include business-performance metrics such as views, watch time, subscriber acquisition, retention, so production decisions should combine these catalog findings with internal performance data.

2 9 . Recommendations

- 1 . Continue growing high-quality TV content. TV Shows have become more prominent among recent releases. Netflix should maintain investment in strong episodic formats.
- 2 . Increase local-language production in priority markets. India and several other non-US markets already have substantial content representation, creating a strong base for further localization.
- 3 . Use country-specific content plans. The ideal mix of Movies, TV Shows and genres should be adapted to each market instead of using one global formula.
- 4 . Keep investing in drama, comedy and international storytelling. These are among the largest categories in the catalog and provide room for both mainstream and local stories.
- 5 . Work with established local actors and directors. Local talent can help make new productions more relatable to regional audiences.
- 6 . Maintain a consistent pipeline of fresh releases. The strong growth in titles added after 2 0 1 6 suggests that a steady flow of new content is central to Netflix's catalog strategy.
- 7 . Consider July, September and December as candidate TV-release windows. These months have high TV addition activity historically, but the launch decision should be validated with actual audience-engagement data.

8 . Use viewer behavior before making large production investments. Combine catalog trends with watch completion rate, subscriber acquisition, retention and revenue data.

3 0 . Executive Summary

The analysis of 8 , 8 0 7 Netflix titles shows that Movies dominate the overall catalog, but TV Shows become increasingly important among recent releases. The dataset contains 6 , 1 3 1 Movies and 2 , 6 7 6 TV Shows. In 2 0 2 1 , TV Shows exceed Movies by release-year count, indicating a meaningful recent shift toward episodic content.

The United States is the largest source of content, while India is the second-largest. This supports a continued focus on international and local-language production. International Movies, Dramas, Comedies and International TV Shows are among the strongest categories, showing the value of combining popular genres with localized storytelling.

Most Movies are around 9 0 – 1 2 0 minutes, while most TV Shows have one or two seasons, with shorter episodic formats an important part of Netflix's catalog. July is the most common month for TV Show additions in the available data, although the dataset cannot establish which month produces the best viewer revenue outcomes.

Overall, Netflix should continue expanding high-quality TV content, strengthen local-language production content mix to individual countries, maintain a steady stream of new releases, and combine catalog analysis with actual viewer and subscriber-performance data before making major production decisions.